Individual Programming Assignment - Song Recommender System
CMPE 256, Spring 2025

Kaushika Uppu
014756859
Kaggle ID: kaushikauppu

RMSE at time of report: 1.7043

Data Preprocessing

Encoding User and Song IDs

The first step of data preprocessing I did was to encode the user and song ids using two separate LabelEncoders, by fitting the encoder with all unique user/song ids in both testing and training sets, and then transforming those variables in each set. I also created a `user_id-song_id` variable in the testing set, which was used as the indices for the submission CSV file.

Song Metadata

Besides the user and song ids, I also included song metadata in the algorithm to try and get better rating predictions. In order to transform the song metadata into something that the model could understand, I used Sentence Transformers[1]. The reason I chose this instead of TF-IDF is because instead of just understanding the importance of a word, I also wanted to embed the semantic meaning of song titles, album names, and artist names, which I think would better aid the model in understanding similarities between certain songs. I also chose Sentence Transformers over word embeddings because I believe that the former would also represent the context of words instead of only the relationship between words, which I think is especially helpful for song metadata (where, for example, different song names could contain the same word with different contextual meanings).

For this process, I first loaded up a pre-trained model, all-MiniLM-L6-v2[2], which although does not have the overall best performance out of all the pre-trained models, is significantly faster, which is important since the training and test sets for this dataset are quite large. Then, using song titles as an example, I got all of the unique song titles in the training set and used the transformer to generate embeddings for them. Since only the unique titles were embedded, I then mapped the titles back to the full training dataset. Sentence Transformers generate dense embeddings, so I then reduced them using PCA to 5, 3, and 3 components for song titles, album titles, and artist names, respectively. These principal components were then added back to the training dataset. This process was carried out for `title`, `release`, and `artist_name`, for both the training and testing sets. The PCA models for each feature were only trained on the training dataset, but were used for transforming both training and testing datasets.

Finally, the last metadata that was included in the model was the year the song was released. Two features were generated from this: `year_norm` and `decade_id`. The first is simply a normalized version of the original year feature, transformed using a MinMaxScaler. The second was created using a LabelEncoder on a calculated `decade` variable, which transformed the decades into categorical IDs 0-9 (1920s-2010s).

---

[1] https://sbert.net/examples/sentence_transformer/applications/computing-embeddings/README.html
[2] https://sbert.net/docs/sentence_transformer/pretrained_models.html

Algorithm and Model

Prior Attempts

I started off this project by trying a base NCF two-tower model with only user and song ids, which resulted in a relatively high RMSE. I used train_test_split to split the training data into training and validation sets since the testing set did not have ratings. I then moved on to trying other models, specifically an XGBoost Regressor model with only users and ids, which performed better than NCF. I then started adding the metadata features explained above to the model, which improved performance slightly, but not significantly. I also tried hyperparameter tuning on the XGBoost model using GridSearchCV, which did decrease the validation set RMSE, but ended up increasing the testing RMSE, which means the model was likely overfitting to the data. Therefore, I decided to try NCF again but with metadata features in an extension of the model we learned in class.

Final Algorithm and Hyperparameter Tuning

The model that ended up giving the best RMSE value had a user tower and song tower, with song metadata being concatenated as well. The `decade_id` was inputted as a separate entity, but after being embedded, was then concatenated to the metadata. In order for the network to learn non-linear and more complex relationships in the data, I used Dense layers with 'relu' activation for the towers. Then, the towers were concatenated and put through two more Dense layers. Finally, the output layer used a 'sigmoid' activation so that all outputs would be between 0 and 1. This is because while experimenting to decrease RMSE, I found that scaling all ratings between 0 and 1 before inputting into the model resulted in lower error. Therefore, before the model was fit to the data, all ratings were scaled using a MinMaxScaler, and were inversely transformed after predictions back to a 1-10 scale. For model compilation, the 'adam' optimizer and 'mse' loss were used, similar to what we did in class. In addition, in order to try and prevent overfitting, I used EarlyStopping[3], which stops training when loss does not improve for a certain number of epochs.

Hyperparameter tuning was also used to find the best parameters for the model. Four parameters were tuned: embedding dimension, number of units for dense layers 1 and 2, and learning rate for the Adam optimizer. KerasTuner's RandomSearch[4] was used for tuning with 5 trials, which resulted in the values [32, 128, 32, 0.001] for the four parameters, respectively. Then, I created `best_model` using the hyperparameters above, and trained and validated using the same data inputs as before. When evaluating the model on the validation set, it was revealed that the tuning did not significantly change the RMSE at all. However, I still used this model to predict ratings for the testing dataset. When submitting to Kaggle, my final RMSE for the testing dataset was 1.7043.

Sources for Code

Code skeletons that were available at the links in the footnotes were used in my code for SentenceTransformer, EarlyStopping, and RandomSearch. A modified version of the code provided in the NCF demo that we did in class was also used in developing the NCF architecture in this project.

---

[3] https://keras.io/api/callbacks/early_stopping/
[4] https://keras.io/keras_tuner/getting_started/#start-the-search